Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım, Uyum ve MVC

Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım, Uyum ve MVC

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kapsamı ve Anlatım Yöntemi
- 2. Kaynaklar ve Ön Koşullar
- 3. Yazılım Kalitesi
- 4. Neden İyi Tasarım? Yazılım Projelerinin Başarısı
- 5. Bağlaşım (Coupling)
- 6. Uyum (Cohesion)
- 7. Tasarım Kalıbı Kavramı
- 8. Örnek Tasarım Kalıbı: Model-View-Controller (MVC)
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Kapsamı ve Anlatım Yöntemi

Dersin ana içeriğini tasarım kalıpları oluşturur. Ders iki kısımdan oluşur (ekrandan, 00:10:00):

- Kısım 1: Gang of Four (Dörtlü Çete) tasarım kalıpları. Dönemin büyük bölümü bu kısma ayrılır.
- Kısım 2: Code Smells and Refactoring (birim sınamaları ile). Bu kısım en fazla üç hafta sürer.

Tasarım kalıpları, nesneye dayalı tasarım ve modelleme etkinliklerinin nasıl doğru yapılacağını anlatır. Kod kusurları (code smells) ise nesneye dayalı modellemede ne gibi yanlışlar yapılabileceğini ve bu yanlışları düzeltmenin yollarını gösterir. Hoca bu ikinci kısmı "hatalardan öğrenme" diye niteledi.

"Dörtlü Çete" adı, dört deneyimli nesne yönelimcinin birlikte yazdığı *Design Patterns: Elements of Reusable Object-Oriented Software* kitabından gelir (Gamma ve diğerleri). Hoca bu kitaptaki kalıpları "zamanın sınavından geçmiş" (time-tested) bir çalışma olarak tanımladı: çok sık karşılaşılan tasarım problemlerinin kaliteli çözümleri. Kitapta 24 kadar tasarım kalıbı anlatılır. Derste, uygulanabilirliğinin düşük olduğunu düşündüğü Interpreter dışındaki bütün kalıplar işlenecektir.

> **Ek bilgi (derste anlatılmadı):** GoF kitabındaki kalıp sayısı büyük olasılıkla 23'tür (5 yaratımsal, 7 yapısal, 11 davranışsal). Hocanın "24 kadar" ifadesi yaklaşık bir sayıdır.

Refactoring kısmında da kod kusurlarının tamamı işlenmez. Hoca, düzeltilmesi daha önemli olan ve daha az çabayla daha çok yarar getiren kusurlardan, kitaptakilerin yaklaşık yarısını oluşturan bir alt küme seçmiştir. Hepsi işlenseydi 14 haftalık döneme sığmazdı.

1.1 Kalıpların anlatılış biçimi

GoF kitabı bir kalıbı bileşenleriyle anlatır. Ana tasarımı bir UML sınıf şeması üzerinden verir, şemadaki sınıfların, metotların ve üyelerin görevlerini açıklar, kalıbın başka hangi kalıplarla işbirliği içinde kullanılabileceğini tartışır. Hoca da bir kalıbı anlatırken önce bu kitaptan alıntılarla kalıbı tanıtır.

Ama bu yetmez. Kalıbın bir problemi çözmek için nasıl uygulandığını ve bu çözümün öteki çözümlerden neden daha iyi olduğunu göstermek gerekir. GoF kitabı kuramsal bir anlatım yapar, uygulamaya az girer. Uygulama kısmı ek kaynak kitaplarda yer alır. Bu kitapların bir kısmı her örnekte bir kalıba odaklanır, bir kısmı daha büyük bir bilgi sistemi üzerinde farklı kalıpları gösterir. Hocaya göre bir ders olarak anlatıldığında "bir örnekte bir kalıp kullanma" yaklaşımı daha rahattır; ders notları da bu biçimdedir. Bazı kalıpların örnekleri bu kitaplardan alınmış, kitaplarda doyurucu örnek bulunamayan kalıplar için örnekleri hoca kendisi hazırlamıştır. Kod kusurları kısmı ise Fowler'ın kitabının bir alt kümesidir.

Hoca, mümkün olduğunda bir kalıbı üç ayrı çözümle örnekleyeceğini söyledi:

1. Akla ilk gelen çözüm.
2. Gereksinimlerde, birinci çözüm üzerinden karşılanması çok verimsiz olacak bir değişiklik gelir. İkinci çözüm nesneye yönelimin ana kurallarına uygun yapılan çözümdür.
3. Uygun bir tasarım kalıbı eklenerek yapılan çözüm. Bunun nesneye yönelik "normal" çözümle karşılaştırılması kalıbın getirdiği kazanımları gösterir.

2. Kaynaklar ve Ön Koşullar

2.1 Kaynak kitaplar

Slayttaki kaynaklar (ekrandan, 00:10:00 ve 00:20:00):

- Kısım 1: *Design Patterns: Elements of Reusable OO Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994.
- Kısım 2: *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999.
- Ek kaynaklar (elektronik ortamda bulunuyorlar):
  - *Design Patterns Explained: A New Perspective on Object-Oriented Design*, Alan Shalloway ve James R. Trott, Addison-Wesley, 2004 (2. baskı).
  - *Head First Design Patterns*, Elisabeth Freeman vd., O'Reilly, 2004.
  - *Design Patterns* (Wordware Applications Library), Christopher G. Lasater, Jones & Bartlett, 2006.
  - *Design Patterns Java Workbook*, S. John Metsker, Addison-Wesley, 2002.
  - *Applied Java Patterns*, S. Stelting ve O. Maassen, Prentice Hall, 2002.

Tasarım kalıpları çok önemli bir konu olduğundan internette de çok sayıda kaynak bulunur. Hoca başka tasarım kalıbı kitaplarından da çalışmanın yararlı olacağını söyledi. Lisansüstü araştırmalar için ACM, IEEE Xplore, ScienceDirect, Springer ve Wiley önemli dergi ve bildiri kaynaklarıdır; Web of Science yalnızca özet taraması yapar; O'Reilly ise daha çok mesleki e-kitaplar için önemlidir.

2.2 Ön koşul 1: Nesneye yönelik programlama

Dersi başarıyla izleyebilmek için herhangi bir güncel nesneye yönelik programlama diline (Java, C++, C#) hakim olmak gerekir. Örnekler Java ile verilir. C# söz dizimi Java'ya çok benzer. C++ bilenler örneklerde biraz zorlanabilir (örneğin kalıtım gösterimi farklıdır), ama karmaşık bir gösterim kullanılmadığı için bu küçük ayrıntılar dışında sorun olmaz.

Tasarım kalıplarının özü temel modelleme işlemleri olduğundan dillere özgü yetenekler kullanılmaz; genellikle interface kavramı bile geçmez. Kalıpta bir soyut sınıf önerildiğinde, alan gerekmiyorsa onun yerine interface kullanılabileceği zaten bilinir.

En temel gereksinim kalıtım ve soyut sınıflardır. Hoca bunu şöyle gerekçelendirdi: önce nesnelerle çalışmak, tasarım yapmak, bunları koda dökmek, hata ayıklamak ve değişikliklerle uğraşmak gerekir. Yani bir işin nasıl yapılacağını bilmek gerekir ki o işin nasıl iyi yapılacağı ya da nasıl yapılmaması gerektiği öğretilebilsin.

Soru: Ders ciddi bir kod bilgisi ve deneyim istiyor gibi görünüyor; “senior” ya da uzman seviyesinde değilsek dersi bırakmak mı daha mantıklı? **Cevap:** Senior olmak gerekmez. Nesneye yönelimin temel kurallarını bilmek ve uygulayabilmek yeterlidir. Örneğin kalıtım hiyerarşisini yanlış kurmak çok şey kaybettirir, ama doğru kullanıldığında kalıtım çok güçlü bir araçtır. Hocanın nesneye yönelik programlama ders notlarında Java’ya özgü kısımlar dışındaki ilk yarıyı (istisna yönetimi dahil) sıkıntısız biliyorsanız ve birkaç ay kod geliştirdiyseniz bu yeterlidir.

2.3 Ön koşul 2: UML sınıf şemaları

İkinci gereksinim UML sınıf şemalarına hakimiyettir. Şemayı çizebilmek, bir şemayı zihinden ya da kâğıt üstünde koda dökebilmek ve bir kodu incelediğinde onun şemasını çizebilmek gerekir. GoF kitabı da ek kaynaklar da tasarımı anlatır ve nesneye dayalı tasarımları görselleştirmenin en iyi standardı UML’dir. Bu derste UML ailesinden özellikle sınıf şemaları, ayrıca sequence diyagramları gerekir. Hoca okların türlerini ve anlamlarını bilmeyenlerin bunu bir UML kitabından bir-iki haftalık çabayla öğrenmesini istedi. Önerilen kaynaklar (ekrandan, 00:20:00): *UML Distilled* (3. baskı, 2003, Martin Fowler, Addison-Wesley) ve *UML 2.0 in a Nutshell* (Dan Pilone ve Neil Pitman, O’Reilly). Head First serisinin kitapları da vardır. UML şemaları artık yazılım belgelendirme etkinlikleri için olmazsa olmaz bir gereçtir.

Derste şemalar Violet UML Editor ile çizildi. Bu araç yalnızca şema çizer, kod dönüşümü yapmaz. Tek bir jar dosyasıdır; Java yüklü olan Windows, Mac ya da Linux makinede çalışır.

3. Yazılım Kalitesi

Tasarım kalıpları üzerine çalışmanın nedeni daha iyi bir tasarım ortaya çıkarmaktır. İyi tasarım iyi bir son ürüne, yani kaliteli yazılıma götürür. Bunun için “iyi” ve “kalite” tanımlanmalıdır (ekrandan, 00:40:00).

Slayttaki tanım şudur: problem alanı hangi düzeyde parçalara ayrılabiliyorsa, o düzeyde parçalara ayrılabilen bir tasarım “iyi” olarak nitelendirilebilir. Hoca bu tanımı güzel, ama öznel buldu. Parçalanma düzeyinin ne olduğu tartışmaya açıktır; daha nesnel ölçütler gerekir.

3.1 Dış ve iç kalite

Kalite ölçütlerine farklı yaklaşımlar vardır. Hocanın tercih ettiği yaklaşım ölçütleri ikiye ayırır (ekrandan, 00:41:40):

- **Dış kalite özellikleri (kullanıcıya yönelik):** doğruluk (programın doğru çalışması), etkinlik (problemi etkin çözmesi), kullanım kolaylığı.
- **İç kalite özellikleri (programcıya yönelik):** yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı.

Tasarım kalıpları iç kalite özelliklerini iyileştirmeye yönelik öğütler verir.

3.2 Yeniden kullanılabilirlik

Yazılımın bazı kısımlarının başka projelerde tekrar kullanılabilmesidir. Maliyeti azaltır. Daha önce kullanılmış ve ayrıntılı test edilmiş kod kullanıldığı için kaliteyi de artırır.

> **Hocanın vurgusu:** Yeniden kullanımdan elde edilebilecek kazanım en fazla %30’lara kadar çıkar; okuduğu araştırmalar hep bu yöndedir. %50-60 gibi bir kazanım beklenmemelidir.

Bunun nedeni, yeniden kullanıma yönelik kodun daha fazla çabayla yazılmasıdır. İlk yazımın maliyeti yüksek olduğundan, bütün harcama kalemlerine bakıldığında sonraki kullanımların getirdiği kazanç çok yüksek kalmaz. Yine de yeniden kullanılabilirlik göz ardı edilmez: “kâr, ne kadar az olsa da kârdır.”

Hoca yeniden kullanım fırsatının beklenmedik yerlerde çıkabileceğini bir örnekle gösterdi. Finans yazılımında döviz kurlarının, sağlık yazılımında hastaların kritik parametrelerinin (örneğin kan değerlerinin) izlenmesi gerekir. İki durumda da izlenen değer belli bir eşiğin altına

ya da üstüne indiğinde ya da bir bandın dışına çıktığında alarm verilir; hekimin ya da yatırımcının dikkati çekilir. İlk bakışta ilgisiz görünen bu iki alanda aynı yapı yeniden kullanılabilir. Bu yüzden yeniden kullanım fırsatlarını sürekli göz açık izlemek yararlıdır.

3.3 İlgi alanlarının ayrılması

Tasarım bileşenlerinin odaklanmış sorumluluklara sahip olması demektir. Odaklanmış sorumluluklar gereksinim değişikliklerinden daha az etkilenir. Ayrıca yeniden kullanılabilirliğe olumlu katkı yapar: odaklanmış bir bileşenin benzer gereksinimli başka projelerde kullanılma olasılığı daha yüksektir.

3.4 Esneklik ve taşınabilirlik

Yazılım alanında çalışanlar bilir ki gereksinimler mutlaka değişir: o alandaki kurallar, kanunlar, yönetmelikler, pazar şartları değişir. Yeterince esnek kod yazılmışsa bu değişikliklere daha rahat uyulur, yoksa yeni baştan projeye başlamak gerekir. Tasarım kalıplarının önemli bir vaadi daha esnek bir tasarımdır.

Taşınabilirlik ise ürünün ne kadar farklı platforma hitap edebildiğidir; ne kadar çok platform, o kadar geniş müşteri kitlesi.

3.5 Okunabilirlik, anlaşılabilirlik ve bakım

> **Hocanın vurgusu:** Okunabilirlik ve anlaşılabilirlik çok önemlidir; yazılımın bakım aşamasında masrafları düşük tutmanın en önemli etkenlerinden biridir.

Yazılımı geliştiren ekip ile yazılım sahaya sürüldükten sonra bakımından sorumlu ekip genellikle farklı insanlardır. Birinin ortaya çıkardığı kodu başka birinin anlayıp ilerletebilmesi için kodun yalnızca okunabilir olması yetmez, anlaşılabilir de olması gerekir. Tasarım kalıpları meslekte çalışanlar arasında ortak bir sözlük oluşturduğu için anlaşılabilirliği de iyileştirir. Okunabilirlik, anlaşılabilirlik ve sınanabilirlik iyileştikçe bakım kolaylığı artar.

4. Neden İyi Tasarım? Yazılım Projelerinin Başarısı

Slayttaki gerekçeler (ekrandan, 00:48:20):

- Yazılım projeleri önemli oranda başarısızlığa uğramaktadır.
- Etken sayısı şüphesiz fazladır, ancak “tasarımın iyiliği” denetlenebilen bir etkendir.
- Oluşan bir hatayı düzeltmenin bedeli, yazılım hayat döngüsünde ilerledikçe üstel olarak artmaktadır.
- Yazılım geliştirme çabalarının çoğu bakım aşamasında harcanmaktadır.

Bu nedenlerle iyi tasarım, başarı ile başarısızlık arasındaki farkı belirleyen önemli bir etkendir.

4.1 Başarının katı tanımı

Bir yazılım projesinin başarılı sayılması için önceden belirlenmiş bütçe aşılmadan, önceden belirlenmiş süre içinde ve gereksinim belgesinde üzerinde anlaşılmış gereksinimlerin tümü gerçeklenmiş olmalıdır. Hoca bunu üç ayaklı bir sehpaya benzetti: üç ayaktan biri bile tutmazsa proje başarısız sayılır.

Hoca bu tanımla ilgili verileri bir bildiri sunumunun slaytlarından gösterdi (ekrandan, 00:50:00). Kaynak *Dr. Dobb’s Journal 2013 IT Project Success Survey* adlı geniş çaplı bir ankettir. Grafikte her süreç türü için başarılı (yeşil), zorlanılmış (sarı) ve başarısız (kırmızı) proje oranları vardır:

Süreç	Başarılı (yaklaşık, ekrandan okunan)
Lean	%70’in biraz üstü
Iterative (yinelemeli)	%65
Agile (çevik)	%65
Ad-Hoc	%50

Süreç	Başarılı (yaklaşık, ekrandan okunan)
Geleneksel (şelale)	%50

Sıralı (şelale) modelle ya da kendi uydurdukları modellerle çalışan firmalarda tam başarı oranı ancak %50'dir. Çevik ve yinelemeli süreçlerde oran %65'e çıkar, Lean geliştirmede biraz daha yükselir. Slayta göre "zorlanılmış" proje bütçe, süre ya da işlev yönlerinden en azından birinde başarısız olan projedir. Başarı tanımı esnetilip bu projeler kısmi başarı sayılırsa oranlar daha da yükselir. Hoca, veri 2013 tarihli olsa da yazılım geliştirme süreçlerinde devrimsel bir değişiklik olmadıkça tablonun aynı kalacağını düşünüyor.

4.2 Türkiye'den bir çalışma

Türkiye'de de durum benzerdir (ekrandan, 00:52:50). UYMS 2014'te yayımlanan "Türkiye'de Çevik Yazılım Geliştirme Üzerine Bir İnceleme" çalışmasında kurumlara, kendi kurumlarında gerçekleştirilen çevik projelerin başarı oranı sorulmuş. 35 katılımcının yanıtları:

Başarılı proje oranı	Katılımcı	Pay
%10'dan az	2	%6
%10-%25	4	%12
%26-%50	5	%14
%50-%75	13	%37
%76-%100	11	%31

Ölçüt "projelerimizin en az %75'i başarılı olsun" diye konursa bu ölçütü karşılayan katılımcı oranı yalnızca %31'dir. Ölçüt %50'ye indirilirse $37 + 31 = 68$, yani %68'dir. Katılımcı sayısı Dobb's anketinden (yüzden fazla) azdır, ama hoca bunu yeterince iyi bir örneklem olarak görüyor.

Hoca bu verilerden şu sonucu çıkardı: süreçler ve modelleme teknikleri iyileşiyor, ama ulaşılması gereken hedef de sürekli yükseliyor. Giderek daha karmaşık, daha büyük ölçekli sistemler gerçekleştirmek gerekiyor; bu yüzden yerimizde saymamak için sürekli koşmak zorundayız. Yapılan iş zordur ve başarıya götürecek her etkene sarılmak gerekir.

5. Bağlaşım (Coupling)

İyi bir tasarıma götüren temel ilkeler slaytta şöyle verildi (ekrandan, 00:54:10):

- Düşük bağlaşım (low coupling)
- Yüksek uyum (high cohesion)
- İlgi alanlarının ayrılması (separation of concerns). Bu ilke ilk iki ilke ile açıklanabilir.

Yani düşük bağlaşım ve yüksek uyuma uyulduğunda ilgi alanları ayrılmış bir tasarıma zaten ulaşılmış olur.

5.1 Tanım

Bağlaşım, bir parçanın diğer parçalara bağımlılık oranıdır (ekrandan, 01:10:00). Parça bir sınıf, bir alt sistem ya da bir paket olabilir; derste özellikle sınıf düzeyi konuşuldu.

Hoca, tanımlanan şeyin kendisinin tanımda geçmesinin iyi olmadığını, ama "bağlaşım" kelimesinin "bağımlılık"tan türetilmiş farklı bir kelime olduğunu belirtti. Bağımlılık şöyle açılır: bir sınıf,

- başka bir sınıfın hizmetlerinden yararlanıyorsa,
- ve/veya onun iç yapısından haberdar olması gerekiyorsa,
- ve/veya onun çalışma ilkelerinden haberdar olması gerekiyorsa,
- ve/veya iki sınıf arasında kalıtım ilişkisi varsa,

o sınıf diğerine bağımlıdır ve aralarında bağlaşım vardır.

Temel düşünce şudur: bir sınıfın ilişkide bulunduğu diğer sınıfların sayısı arttıkça bağlaşımı yükselir. Bağlaşım düşük tutulursa bir sınıfta yapılan değişiklik sistemin geri kalanında daha az sınıfı etkiler. Bu da daha esnek bir tasarım sağlar ve yeniden kullanılabilirliği yükseltir.

5.2 Bağlaşım nasıl sayılır?

Hoca Violet UML'de üye alanları ve metotları olmayan A, B, C sınıfları çizdi. Bağlaşım sınıfların, daha doğrusu bu sınıflardan oluşan nesnelerin birbiriyle ilişkisidir; gösterim ise sınıflar üzerinden yapılır.

"A, B'ye bağımlıdır" demek, örneğin A sınıfında B türünden bir ya da daha çok üye bulunması demektir. İlişkinin niceliğini okun ucuna yazılan çokluk göstergesi belirler (örneğin bir A nesnesi sıfır ya da daha fazla B nesnesiyle ilişkili olabilir). Bağlaşım açısından önemli olan ilişkinin varlığıdır, niceliği değildir. A'nın B'ye "0..*" bağımlılığı ile B'nin C'ye "1" bağımlılığı arasında bağlaşım hesabında fark yoktur; ikisi de birer ilişki sayılır.

Okların yönü ise önemlidir (02:31:47'de yapılan ek açıklama). A, B'ye bağlıdır ama B, A'ya bağlı değilse yalnızca bir ilişki sayılır. B de A'ya bağlı olsaydı (iki yönlü ok) toplam bağlaşım bir artardı.

5.3 Zincir mi, yıldız mı?

Aynı problemi çözen iki tasarım verildi (ekrandan, 01:20:00):

```
Tasarım I (zincir):    A ---> B ---> C
Tasarım II (yıldız):   A ---> B
                       A ---> C
```

Hoca hangisinin daha düşük, yani daha iyi bağlaşıma sahip olduğunu el kaldırma ile oylattı: 5 kişi Tasarım I'i, 9 kişi Tasarım II'yi seçti. Ardından tek tek sınıfların bağlaşımı hesaplandı (ekrandan, 01:30:00).

Doğrudan ilişkilerle:

- Tasarım I: bağ(A) = 1 (B'ye bağlı), bağ(B) = 1 (C'ye bağlı), bağ(C) = 0.
- Tasarım II: bağ(A) = 2 (B'ye ve C'ye bağlı), bağ(B) = 0, bağ(C) = 0.

Tek tek sayılar tek başına çok şey söylemez. Sistem düzeyinde karar vermek için iki ölçü düşünülebilir: **toplam bağlaşım** ve **maksimum bağlaşım**.

- Toplam: Tasarım I için $1 + 1 + 0 = 2$, Tasarım II için $2 + 0 + 0 = 2$. Eşit çıktılar; bu ölçü karar verdirmedi.
- Maksimum: Tasarım I için $\max(1, 1, 0) = 1$, Tasarım II için $\max(2, 0, 0) = 2$. Bu ölçüye göre Tasarım I daha iyi.

Maksimuma bakmanın gerekçesi oyun kuramından gelir. Seçeneklerin hepsi kötüyse, olası en kötü senaryolar listelenir ve bunlardan en az zarar riskli olan seçilir. Bağlaşımın düşük olmasını istediğimiz için en yüksek bağlaşımı daha düşük olan sistem seçilir.

> **Ek bilgi (derste anlatılmadı):** Bu karar kuralı karar kuramında büyük olasılıkla "minimaks" (en kötü durumdaki kaybı en aza indirme) diye bilinir. Ölçütün en kötü sınıfa odaklanması, sistemde değişikliklerden en çok etkilenecek tek bir sınıfın oluşmasını engellemeyi amaçlar.

Dolaylı ilişkilerle: "Şeytan ayrıntıda gizlidir." Tasarım I'de A, B üzerinden C'ye dolaylı olarak da bağlıdır. Bir ölçüt kullanırken dolaylı ilişkilerin hesaba katılıp katılmayacağına ve hangi ağırlıkla katılacağına karar vermek önemlidir. Hoca dolaylı ilişkiye 0,5 ağırlık verdi:

- Tasarım I: bağ(A) = 1 + 0,5 = 1,5; bağ(B) = 1; bağ(C) = 0. Toplam 2,5, maksimum 1,5.
- Tasarım II: değişmez. Toplam 2, maksimum 2.

Ekrandaki notlarda daha iyi değerler "+" ile işaretlendi: toplamda Tasarım II ($2 < 2{,}5$), maksimumda Tasarım I ($1{,}5 < 2$) üstündür. Resim belirsizleşti.

Neden 0,5? Dolaylı ilişkiye doğrudan ilişki kadar ağırlık vermek doğru olmaz. C'nin kaynak kodunda bir sorun düzeltilirken yapılan değişikliğin B'yi doğrudan etkileme olasılığı yüksektir, A'ya kadar taşınma olasılığı daha düşüktür. Bu yüzden yarım ağırlık vermek hocaya göre yeterli ve daha doğru bir davranıştır.

5.4 Hangi ölçüt seçilmeli?

Bu tartışma kesin bir karara götürmedi; hoca kararın nasıl verileceğini anlattı. Toplam bağlaşıma mı maksimum bağlaşıma mı göre hareket edileceğine karar vermek için geçmiş yazılım projelerine bakılır:

1. Projeler için bir başarı ölçütü belirlenir. Örneğin bütçe, süre ve işlev ayaklarından ulaşılanlara 1, ulaşılamayanlara 0 verilir; ya da projedeki toplam hata düzeltme çabası ne kadar azsa proje o kadar başarılı sayılır. Yazılım evi neye odaklanıyorsa ona göre bir ölçüt seçilir.
2. Başarılı projelerde toplam bağlaşımın mı yoksa maksimum bağlaşımın mı daha iyi çıktığına bakılır.
3. Önem verilecek ölçüt buna göre seçilir.

Bu masraflı bir iştir. Tamamlanan projelerin kayıtları sağlam tutuluyorsa ve bağlaşım, uyum gibi yazılım ölçütlerini otomatik hesaplayan araçlar varsa kolaylaşır.

5.5 Problem alanı her şeyden önce gelir

Hoca, öğrencilerin şu itirazı yapabileceğini söyledi: "A ne, B ne, C ne? Bu varlıklar problem alanında neyi ifade ediyor? Gerçekten aralarındaki ilişki birinci sistemdeki gibi mi, ikincideki gibi mi?" Bu itiraz haklıdır. Gerçek problem alanında varlıklar arasındaki ilişkiler nasılsa tasarımdaki ilişkiler de öyle kurulmalıdır. Hangi tasarım seçilirse seçilsin gerçek hayata uymayan bir model kullanılmamalıdır; ilk ölçüt budur. Bağlaşım tartışmaları, sınıflar tasarımcının serbestçe karar verebileceği, örneğin sistem ya da altyapı katmanında, problem alanıyla doğrudan ilişkili olmayan sınıflar olduğunda geçerlidir.

6. Uyum (Cohesion)

6.1 Veritabanı sorumluluğu kimde olmalı?

Hoca kolaylık için Tasarım I'i (A→B→C) seçti ve yeni bir durum ekledi: bu sınıflar problem alanı varlıklarıdır ve bir veritabanında (ilişkisel ya da değil) saklanmaları gerekir. İki tasarım çizildi (ekrandan, 01:40:00):

```
Tasarım I (VT işlemleri sınıflarda kodlu):
  A ---> B ---> C
  A <--«CRUD»--> [Veritabanı]
  B <--«CRUD»--> [Veritabanı]
  C <--«CRUD»--> [Veritabanı]

Tasarım II (VT işlemleri ilgili tek sınıfta kodlu):
  A ---> B ---> C
  VT - - -> A,  VT - - -> B,  VT - - -> C
  VT <--«CRUD»--> [Veritabanı]
```

Tasarım I'de A, B ve C nesneleri kendi okuma, yazma ve ilklendirme işlerini (create, read, update, delete) kendileri yapar. Tasarım II'de bu işlerin tamamından ayrı bir VT sınıfı sorumludur. VT sınıfının bu işleri yapabilmesi için A, B, C ile bağımlılık ilişkisi kurması gerekir: sahiplik (üye alan) şeklinde olmasa da bu türden nesneleri oluşturmak, parametre olarak kullanmak ya da geri döndürmek için. Oylamada 1 kişi Tasarım I'i, 13 kişi Tasarım II'yi seçti.

Hoca şunu sordu: madem bağlaşımın düşük olması gerektiğini söyledik, yeni VT sınıfı sistemin bağlaşımını önemli ölçüde artırıyor. Buna rağmen neden Tasarım II?

Soru: Neden ikinci tasarımı seçtiniz? **Cevap (bir öğrenci):** İkinci sistemde veritabanı bağlantısı tek bir yerden yapılır. İleride bunu değiştirmek istediğimde tek yerden değiştirmem yeter;

birinci sistemde birçok yerde değişiklik gerekir. İkinci sistemle daha maliyet etkin bir çözüm elde edilir. Hoca bu yanıtı onayladı.

Soru (Tasarım I'e oy veren bir öğrenci): İkinci sistemde VT sınıfına çok fazla sorumluluk yüklüyoruz; hem çok bağımlılığı var hem de A'nın, B'nin ve C'nin işlemlerini birlikte yapıyor. Birinci sistem daha sağlıklı olmaz mı? **Cevap:** Hoca bir benzetmeyle yanıt verdi: bir babanın bir çocuğunun olması ile beş çocuğunun olması arasında sorumluluk açısından büyük fark vardır; her çocuk daha fazla çaba ve maliyet demektir. Yazılımda durum böyle değildir. VT sınıfı bu işi bir sınıf için yapıyorsa, iş mantığı ve kodu aynıdır; aynı şeyi beş, on ya da elli kez yapar. Bu nedenle VT sınıfının sorumlulukları birbiriyle uyumludur.

6.2 Uyumun tanımı ve benzetmeler

Uyum, bir parçanın sorumluluklarının birbiriyle uyumlu olma derecesidir. Burada bakılan, parçanın yürüttüğü rol ya da yaptığı iştir. Hocanın benzetmeleri:

- Bir yazılım evinde test mühendisi olarak çalışırken size aynı zamanda geliştirici görevleri de verilirse hangi sorumluluğu hangi sırada yerine getireceğinizi şaşırırsınız.
- Bir montaj hattında biri kaporta, biri motor, biri elektrik hattında çalışır. Her işi bilen çok yetenekli işçiler olsa bile, her işçinin kendi uzmanlaştığı işi yaptığı hat çok daha hızlı çalışır.

Tasarım I'de A, B, C sınıfları zaten kendi iç mantıklarıyla ilgili sorumluluklar taşır; üstüne veritabanı sorumluluğu eklenince bu varlıklara fazladan sorumluluk yüklenmiş olur. Tasarım II'de A, B, C yalnızca iş mantığıyla uğraşır, VT sınıfı yalnızca veritabanı kalıcılığıyla uğraşır. VT sınıfında birbiriyle uyumlu işler yapan, odaklanmış metotlar bulunur. Bu nedenle Tasarım II uyum açısından daha iyidir.

> **Ders sonrası not:** Hoca 01:43:09'da "iki numaralı tasarım bağlaşım açısından daha iyidir" dedi. Bağlamdan anlaşıldığına göre kastedilen uyumdur: Tasarım II'nin bağlaşımı daha yüksek, uyumu daha iyidir.

6.3 Katmanlı yapı

Hoca ekran üstüne çizerek Tasarım II'deki yapıyı gösterdi: A, B, C sınıfları iş mantığı katmanını, VT sınıfı ve veritabanı ise veritabanı katmanını oluşturur. Böylece:

- İş mantığında yapılacak değişiklikler veritabanı katmanını etkilemez.
- Veritabanı değiştirildiğinde, örneğin Microsoft SQL Server ya da Oracle'dan MySQL gibi ücretsiz bir veritabanına geçildiğinde, hatta ilişkisel olmayan (NoSQL) bir veritabanına geçildiğinde, veritabanı işlemleri çok farklı kodlanmak zorunda kalır. Bu kod değişikliklerinin hiçbiri iş mantığı sınıflarını etkilemez.

Maliyet etkinliği getiren şey de aslında uyumun yüksek olmasıdır.

6.4 Yüksek uyumun yararları

- Sınıf odaklanmış bir iş yaptığı için anlaşılması kolaydır.
- Odaklanmış iş başka projelerde daha rahat yeniden kullanılır.
- Sınıfın değişikliklerden etkilenme olasılığı düşer; bu doğrudan bakım kolaylığı getirir.
- Uyum düşükse, yani sınıfa farklı farklı sorumluluklar yüklenirse kaynak kodu genişler: daha fazla metot, daha uzun ve karmaşık bir sınıf ortaya çıkar ve anlaşılması zorlaşır.

6.5 Bağlaşım ve uyumun ilişkisi

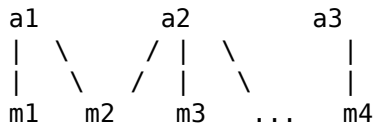
Genel olarak düşük bağlaşım getiren bir tasarım yüksek uyumu da beraberinde getirir; yüksek bağlaşım getiren bir tasarım da düşük uyumu getirir. Bu yüzden iki ilke çoğunlukla birbiriyle bağdaşır ve dikkatli olmak kaydıyla bağlaşıma göre karar vermek uyuma göre de karar vermek olur.

Ancak derste yapılan veritabanı örneğinde olduğu gibi, az sayıda ama önemli durumda iki ilke çatışır.

> **Hocanın vurgusu:** Bağlaşım ile uyum çeliştiğinde hangi tasarım daha katmanlı, daha derli toplu bir çözüm oluşturuyorsa o seçilmelidir. Bu durumlarda uyuma daha çok dikkat etmek gerekir.

6.6 Uyumun ölçülmesi: metot-alan erişim deseni

Uyumu hesaplamak için farklı yöntemler vardır. Hocanın daha kolay anlattığı yöntem, sınıf içindeki üye alan ve üye metot erişim desenini çıkarmaktır. Örnek sınıfta üstte üç alan, altta dört metot vardır (ekrandan, 01:50:00):



`Erişimler: m1→a1, m1→a2, m2→a1, m2→a2, m3→a2, m4→a3`

Ölçüt, mevcut erişim sayısının "tam bağlı" (her metodun her alana eriştiği) durumdaki en çok erişim sayısına oranıdır:

- Olası en çok erişim: 3 alan $\times$ 4 metot $= 12$.
- Gerçekleşen erişim: 6.
- Uyum: $6/12 = 0{,}5$, yani %50. %0 çok kötü, %100 çok iyi, %50 ortalama sayılabilir.

Uyumu iyileştirmek: Hoca sınıfı a2 ile a3 arasından ikiye böldü (ekrandan, 01:51:10):

- Sınıf I = {a1, a2; m1, m2, m3}. Olası en çok erişim $2 \times 3 = 6$, gerçekleşen 5 (m1→a1, m1→a2, m2→a1, m2→a2, m3→a2). Uyum $5/6 \approx 0{,}83$.
- Sınıf II = {a3; m4}. Olası en çok erişim $1 \times 1 = 1$, gerçekleşen 1. Uyum $1/1 = 1$, yani %100.
- Ortalama uyum: $(5/6 + 1)/2 \approx 0{,}917$.

Tek sınıfta %50 olan uyum, sınıf iki ayrı sınıfa bölündüğünde ortalama yaklaşık %92'ye çıktı.

> **Ders sonrası not:** Hoca ortalamayı yuvarlanmış %83 üzerinden "183 bölü 2, 91,5" diye hesapladı. Tam değerle $(83{,}33 + 100)/2 \approx 91{,}67$, yani yaklaşık %91,7'dir.

Bu tür sayısal değerler tasarımı değiştirme kararlarına dayanak olabilir. Mevcut tasarımı, dışarıya verilen hizmetleri değiştirmeden iç tasarımını değiştirerek iyileştirme çalışmasına **refactoring** denir; dersin son haftalarının konusu budur.

Uyum hesabı için başka yöntemler de vardır: koda konan yorumlardan yola çıkan, makine öğrenmesine dayanan teknikler gibi. Ama erişim deseni üzerinden hesap daha pratiktir ve anlatması daha kolaydır.

7. Tasarım Kalıbı Kavramı

7.1 Tanım ve amaç

Slayttaki tanım (ekrandan, 02:10:00): tasarım kalıbı, yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Çözüm somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir. GoF kalıplarının çoğu somut, yani epey ayrıntılıdır. Daha az ayrıntılı bir kalıp olarak Model-View-Controller ile başlanacaktır.

Amaç tekerleği yeniden keşfetmeden "iyi" tasarıma ulaşmak, sorunları esnek biçimde ve yeniden kullanılabilirliği artıracak yönde çözmektir.

7.2 Bir tasarım kalıbının dört bileşeni

1. **İsim:** Kalıbı en güzel şekilde anlatacak bir ya da birkaç kelimelik isimdir. İyi isim vermek sanıldığından çok daha zordur. Kalıp incelenip özümsendikten sonra yalnızca adı söylendiğinde öteki üç bileşenin ne olduğunu hatırlatabilmelidir. Hocaya göre Dörtlü Çete kalıplarına çok güzel isimler vermiştir.
2. **Sorun:** Kalıbın çözdüğü sorunun tarifidir.

3. **Çözüm:** Sorunu çözmek için önerilen tasarımın bileşenleri, bu bileşenlerin ilişkileri, sorumlulukları ve birlikte çalışmalarıdır. UML sınıf şeması, gerekirse başka şemalar ve kodla anlatılır.
4. **Tartışma/Sonuçlar:** Sonda olması önemsiz olduğu anlamına gelmez. Mükemmel çözüm yoktur; bu bölümde çözümün getirileri ve götürüleri tartışılır: sistemin geneline esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden etkileri, güçlü ve zayıf yönleri, yer-zaman çelişkileri ve başka kalıplarla işbirliği olanakları.

Yer-zaman çelişkisi: Bir problemi çözen kod ya kullandığı bellek açısından ya da çalışma süresi açısından optimize edilir. Birini iyileştirmek ötekinde masrafı artırır: daha fazla bellek harcayan kod daha hızlı çalışabilir, bellekten tasarruf etmek gerekirse daha fazla hesaplama yapılır (time-memory trade-off). Her yönden en iyi çözüm yoktur; tartışma bölümü dengenin nerede kurulacağına dair öneriler verir.

7.3 Kalıplar arası ilişkiler

GoF kitabındaki bir şema, kitaptaki bütün kalıplar arasındaki olası ilişkileri gösterir (ekrandan, 02:13:10). Şemadaki bazı ilişkiler:

- Template Method → Strategy (“defining algorithm’s steps”): şablonun alt adımlarının her biri birer Strategy üzerinden gerçeklenebilir.
- Composite → Iterator (“enumerating children”), Composite → Visitor (“adding operations”), Decorator → Strategy (“changing skin versus guts”).
- Abstract Factory → Singleton ve Facade → Singleton (“single instance”), Abstract Factory → Factory Method (“implement using”), Abstract Factory → Prototype (“configure factory dynamically”).
- Observer → Mediator (“complex dependency management”).

Hoca şemada olmayan bir ilişkiyi de ekledi: Strategy → Singleton. Strategy gerçeklemelerinin bazılarının Singleton olma olasılığı vardır; bu, GoF’un Strategy anlatımında sözle geçer ama şemaya çizilmemiştir.

> **Hocanın vurgusu:** Bu şemayı kesinlikle ezberlemeniz gerekmiyor. Yalnızca kalıpların ne ölçüde birbirine bağlı olduğu hakkında görsel bir fikir verir.

Şemanın bir yararı şudur: bir kalıbı kullanırken problemin başka kısımlarına başka kalıplar eklenip eklenemeyeceği düşünülüyorsa (örneğin bir Composite’in elemanları dolaşılacaksa Iterator kullanılabilir), bunu bilerek o gözle bakmak, sıfırdan araştırmaktan daha kolaydır.

7.4 Tasarım kalıplarının kazanımları

- İşler yeniden icat edilmez; aynı problemin çözümü için sıfırdan düşünüp taşınmak gerekmez.
- Ortaya çıkan çözüm, nesneye yönelimde deneyimli birçok kişinin emeğinin geçtiği kaliteli bir çözümdür.
- Deneyimin yazılı biçimde aktarılmasını sağlar. Nesneye yönelimin temellerini bilen ama modellemede çok deneyimli olmayan biri bile, bir öğretmene ihtiyaç duymadan kalıplar üzerine çalışarak deneyim kazanabilir. Bu yüzden konu kendi kendine çalışmaya çok açıktır.
- Yazılım ekibi arasında ortak bir sözlük oluşturur.

Hoca ortak sözlüğü bir örnekle anlattı. Bir projede gereksinim belgesinden yola çıkarak tasarım yapan birkaç kıdemli yazılım mimarı düşünelim. Biri “Sistemde farklı optimizasyon algoritmaları kullanmamız gerekiyor; bunların her birini Strategy olarak gerçekleyebiliriz” der. Öteki “Çok güzel. Öte yandan bazı algoritmaların alt adımları zamanla değişebilir; bu algoritmaları Decorator şeklinde yaparsak daha rahat değiştirebiliriz” der. Birkaç kelimeyle çok önemli tasarım ayrıntılarına karar verilmiş olur. Tasarımı gerçekleyecek ekip de kalıpları bildiğinde bütün ekip ortak bir sözlüğe sahip olur ve birlikte çalışmanın verimi artar.

Bu nedenlerle sürecin neresinde olursa olsun her yazılımcının kalıpları bilmesi gerekir. Teknik olmayan bir proje yöneticisinin bilmesi gerekmeyebilir, ama işin teknik kısmına dokunan herkesin bilmesinde çok yarar vardır. Biri bir kalıbı çok temel bir ilke gibi de görebilir; hoca buna itiraz etmedi, ama bazen tek bir fikrin bile çok önemli noktalara götürebileceğini ekledi.

7.5 Kalıpların sınıflandırılması ve anlatım sırası

Ders notlarında GoF kalıpları kaynak kitaptaki sıralamaya sadık kalınarak anlatılır. Kalıplar amaçlarına göre üç kümeye ayrılır (ekrandan, 02:20:00):

- **Creational (yaratımsal):** Nesnelerin oluşturulması, temsili ve ilişkilendirilmelerindeki bağımlılığı azaltmaya yönelik kalıplardır. Yani nesne oluşturma, temsil ve ilişkilendirmeyi en az bağlaşımla yapmanın yollarını verirler.
- **Structural (yapısal):** Sınıflar ve nesnelerin bir araya getirilerek daha büyük yapılar elde edilmesine yönelik kalıplardır.
- **Behavioral (davranışsal):** Sorumlulukların nesnelere dağıtılması ve algoritma seçimine yönelik kalıplardır.

Her kalıp yalnızca bir kümeye konur. Ayrıca kapsama göre ikinci bir sınıflandırma vardır:

- **Sınıfsal kalıplar:** Sınıflar ve kalıtımla oluşturulan alt sınıflar arasındaki ilişkilere yönelik öneriler getirir.
- **Nesnesel kalıplar:** Çalışma anında nesneler arasında kurulan, değişen ve bozulan ilişkilere yönelik, koda daha dinamik açıdan bakan kalıplardır.

Adapter hem sınıfsal hem nesnesel olarak değerlendirilir ve kitaptaki tabloda iki yerde geçer; diğer bütün kalıplar yalnızca birine konur. Hoca MVC'nin de iki farklı kategoriye konabileceğini söyledi.

GoF kitabının giriş bölümünde bir de kalıpların hangi tasarım problemini çözdüğüne göre bir gruplama vardır:

- Uygun nesneleri bulmak (örneğin Composite, Strategy, State).
- Nesne taneciklilik (granularity) düzeyini belirlemek: bütünün daha ince ya da daha kalın parçalardan oluşturulması ve doğru ölçünün bulunması.
- Nesne arayüzlerini belirlemek (örneğin bazı niteliklerin gizlenmesine yönelik Memento; Decorator, Proxy, Visitor).

Hoca birkaç yıl kalıpları bu amaca yönelik gruplamaya göre anlatmış. Sonra bu sırada bir kalıptan başkasına atıf yaparken ya da benzerlikleri ve farkları anlatırken henüz görülmemiş, hatta uzun süre görülmeyecek kalıplara çok sık değinmek zorunda kaldığını fark etmiş. Dörtlü Çete de esas anlatım sırası olarak yaratımsal-yapısal-davranışsal sırasını seçtiğine göre "bir bildikleri vardır" diyerek kitabın sırasına dönmüş.

> **Hocanın vurgusu:** Kalıplar arasında yüzeysel olarak benzer ama nitelik olarak farklı olan ayırt edici noktaları bilmek, doğru kalıbı seçebilmek için çok önemli bir ölçüttür. Bunlar yeri geldikçe vurgulanacaktır.

8. Örnek Tasarım Kalıbı: Model-View-Controller (MVC)

Kalıbın adına ilk bakışta bir şey anlaşılmaz, ama kalıp çalışıldıktan sonra geriye dönüp bakınca "ismiyle müsemma" olduğu görülür.

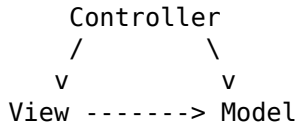
8.1 Sorun ve çözüm

Slayttaki tanım (ekrandan, 02:30:00 ve 02:31:10):

- **Sorun:** Aynı veriyi farklı biçimlerde gösterebilmek istiyoruz. Kullanıcıya verinin nasıl ve ne kadarının gösterileceğini seçebilme gibi olanaklar sağlamak istiyoruz.
- **Çözüm:** Veriyi, verinin gösterimini ve gösterimin yönetimini ayrı sınıflar şeklinde ele almak.
  - **Model sınıfı:** Ham veriyi simgeler.
  - **View sınıfı:** Verinin çeşitli gösterim biçimlerini simgeler.
  - **Controller sınıfı:** Kullanıcı komutlarını alıp işler; örneğin belli bir anda hangi gösterim biçiminin istendiğini belirler.

Bu, iyi tasarım tanımının doğrudan uygulamasıdır: ihtiyaç hangi düzeyde sorumluluk kırılımına sahipse çözüm de o düzeyde parçalanır. Verinin kendisi, gösterim biçimi ve gösterimin düzenlenmesi ayrı esneklikler istendiği için birbirinden ayrılır.

8.2 Sınıflar arasındaki ilişkiler



Bağlaşımı gereksiz yere artırmamak için yalnızca ihtiyaç olan yönde bağımlılık kurulur:

- **Model** ne nasıl gösterildiğini ne de kimin tarafından denetlendiğini bilmek zorundadır. Model yalnızca veridir; public metotlarını kim çağırırsa (bunu bilemez de) ona ilgili hizmetleri vermekle yükümlüdür.
- **View → Model:** View hangi veriyle çalıştığını bilmek zorundadır; ilgili çizimi yaparken gereken verileri modelden alır.
- **Controller → View ve Controller → Model:** Controller bir view oluştururken ona "kullanacağın veri budur" diye veriyi verebilmelidir; bu yüzden ikisini de bilir.

Bu üç ilişki asgari ilişkilerdir. Bu minimum ilişkiler işi görüyorsa burada bırakılmalıdır.

8.3 Adın kaynağı: ressam benzetmesi

Bir ressam bir insan resmi yaparken poz veren kişiye "model" denir. Diyelim ki biri resmini bir sanatçıya yaptırmaya karar verdi. Veri, resmi yapılacak kişidir. Bu kişi üzerinde farklı bakış açıları olabilir: boy resmi ya da yalnızca omuzlardan yukarısını gösteren portre; yağlıboya ile gerçekçi renkli ya da karakalem; ya da yüz hatlarının ve vücudun abartıldığı bir karikatür. Bunlar view'lardır. Kişiyi çizen ressam ise kontrolcüdür: esas çizimi o yapar ve verdiği sanatsal kararlar doğrultusunda ilgili view'dan yola çıkarak modeli çizer. Bu benzetme akılda kaldığında MVC'nin işi ve kalıbın bütünü hemen hatırlanır. Hoca öğrencilerin bulacağı başka benzetmeleri de duymak istediğini söyledi.

8.4 Yararlar

Slayttaki yararlar (ekrandan, 02:40:00): gösterim, kontrol ve veriyi birbirinden ayırarak

- bunları çalışma anında değiştirebilmek (örneğin bir gösterimden başka bir gösterime geçmek),
- ilgi alanlarının ayrılmasını sağlamak (sorumluluklar ayrıldığı için bağlaşım düşer, uyum yükselir),
- yeniden kullanımı artırmak.

8.5 Çeşitlemeler

Bu tasarım üzerine ek ihtiyaçlar gelebilir; tasarım bu ihtiyaçlar doğrultusunda değiştirilir (ekrandan, 02:44:10).

1. Model'in geçirdiği değişiklikleri View'a bildirmesi. Temel tasarımda Model'in View'a bir şey bildirmesi gerekmez: View modeli bir kez alır ve çizer. Ama veri durağan değilse ve sürekli güncelleniyorsa durum değişir. Hocanın örneği seçimlerdir: her partinin her il ve ilçede aldığı oy oranları bazen çubuk grafikle, bazen pasta grafikle, bazen harita üzerinde, bazen ilçe, bazen büyükşehir kırılımında gösterilir. Yeni sandıklar açıldıkça veri sürekli değişir ve güncel sonuçlar olabildiğince kısa sürede ekranda görülmek istenir.

En iyi yanıt süresi için Model değişikliklerini View'a bildirmelidir. Bunun için Model'den View'a doğru bir ok eklenir: Model hangi view tarafından çizildiğini bilir ve ona "ben değiştim, beni yeniden çiz" der. Bu noktada daha sonra görülecek Observer kalıbı devreye girer; Model ile View arasında Observer kurulabilir. Bu, MVC ile Observer arasındaki ilişkidir.

> **Ek bilgi (derste anlatılmadı):** Observer kalıbında büyük olasılıkla gözlenen nesne (subject) kendisini izleyen gözlemcilerin listesini tutar ve durumu değiştiğinde hepsine bildirim gönderir. Gözlemciler soyut bir arayüz üzerinden tanındığı için Model, View'ların somut sınıflarına bağımlı olmaz.

Bu ok eklenmek istenmezse View'un modeli belirli aralıklarla sorgulaması ve değişiklik varsa yeni veriyi alıp çizmesi gerekir. Burada sorgulama sıklığı önemlidir: çok seyrek sorgulanırsa

son değişikliklerden haberdar olunmaz; çok sık sorgulanırsa ağ trafiği gereksiz yere artar. Model ile View farklı bilgisayarlarda, hele aynı yerel ağda değil farklı yerlerdeyse iletişim maliyeti daha da büyür. Uygun denge farklı problem alanlarına ve isterlere göre farklı yönde kurulur. Kısacası bağlaşımı yükseltmemek için bu ok çizilmezse, bunun bedeli haberleşme maliyeti ya da güncellikten ödündür.

2. Birden fazla view. Bazı view'lar şöyle olabilir:

- **İç içe (nested):** Marmara bölgesindeki illerin oy oranları gibi: ilçeler ilin, il bölgenin, bölge ülkenin içindedir.
- **Ayrı/tek görevcikte (thread):** Belki ayrı bir görevcik modeli sürekli güncellerken view, bilgisayardaki pencere yöneticisinin ortamında çalışır; buna göre kararlar vermek gerekir.
- **Örtüşen/ayrık:** Bir PDF okunurken her "page down" ile ekranda tam bir sayfa görünebilir (ayrık); ya da ekranın bir kısmında önceki sayfanın sonu, bir kısmında sonraki sayfanın başı görünebilir (örtüşen).

Birden fazla view varsa ve bir model aynı anda bir ya da daha fazla view tarafından gösteriliyorsa (örneğin o anda etkin olan view kavramı varsa), Model→View ilişkisi bire-bir değil bire-çok olur. Çokluk "*" olduğunda bu ilişkiyi bir dizi, liste ya da ağaç gibi bir veri yapısıyla modellemek gerekir; bu da ihtiyaçlar doğrultusunda daha karmaşık kod demektir.

3. Birden fazla kontrolcü. Örneğin bir üç boyutlu grafik düzenleyicide ya da bir metin düzenleyicide kullanıcı girdisi klavyeden, fareden ya da sesle dikteyle yapılabilir. Kontrolcüler de birden fazla olabilir. Bu kontrolcüler arasında çalışma anında geçiş yapmak isteniyorsa, daha sonra görülecek Strategy kalıbını kullanmak yerinde olabilir.

Bütün bu konularda karar verilmesi gerektiğinin farkında olmak, üzerinde çalışılan kodu yazarken önemlidir.

8.6 MVC kalıbı ve MVC çatısı

Hoca bir karışıklığa dikkat çekti: MVC denince Microsoft'un MVC çatısı (framework) akla gelebilir. MVC'nin ilk ve özgün kullanımı bir tasarım kalıbı olarak kullanımıdır. Microsoft'un MVC çatısı, bu kalıbı burada konuşulan ayrıntılarla ve daha fazlasıyla, özellikle .NET ortamında geliştirilen projelerde rahatça kullanmayı sağlayan yetenekli bir çatıdır.

Hoca, yazılım geliştirme projelerinde çalışan öğrencilerden bu kalıbı (ya da ileride görülecek kalıpları) nerede, nasıl, hangi amaçla ve hangi çeşitlemelerle kullandıklarını paylaşmalarını istedi. Belki kalıbın varlığını bilmeden tekerleği yeniden keşfetmiş olanlar da vardır; bir varyasyonun işe yarayıp yaramadığını duymak herkes için yararlıdır.

Örnek Sorular

Soru 1: Hangi tasarımın bağlaşımı daha düşük? (ekrandan, 01:20:00 ve 01:30:00)

Aynı problemi çözen iki tasarım veriliyor. Tasarım I: A→B, B→C. Tasarım II: A→B, A→C. Bağlaşım açısından hangisi tercih edilmelidir? Önce yalnızca doğrudan ilişkilerle, sonra dolaylı ilişkilere 0,5 ağırlık vererek toplam ve maksimum bağlaşımı hesaplayın.

Derste verilen çözüm:

- Doğrudan: Tasarım I'de bağ(A) = 1, bağ(B) = 1, bağ(C) = 0; toplam 2, maksimum 1. Tasarım II'de bağ(A) = 2, bağ(B) = bağ(C) = 0; toplam 2, maksimum 2. Toplam eşit, maksimuma göre I daha iyi.
- Dolaylı (0,5): Tasarım I'de bağ(A) = 1,5; toplam 2,5, maksimum 1,5. Tasarım II değişmez. Toplamda II (2), maksimumda I (1,5) daha iyi.
- Kesin karar için geçmiş projelerin başarı verisiyle hangi ölçütün başarıyla daha iyi örtüştüğüne bakılır. Her durumda önce gerçek problem alanındaki ilişkilere uygun model kurulmalıdır.

Soru 2: Veritabanı sorumluluğu kimde olmalı? (ekrandan, 01:40:00)

A→B→C tasarımındaki varlıklar bir veritabanında saklanacaktır. Okuma-yazma (CRUD) sorumluluğu A, B, C sınıflarının her birine mi verilmeli (Tasarım I), yoksa bu işler ayrı bir VT sınıfında mı toplanmalı (Tasarım II)? Tasarım II bağlaşımı artırdığı halde neden tercih edilebilir?

Derste verilen çözüm:

- Tasarım II tercih edilir. Bağlaşım artar, ama uyum yükselir: A, B, C yalnızca iş mantığıyla, VT sınıfı yalnızca kalıcılıkla uğraşır.
- Veritabanı bağlantısı tek yerdedir; veritabanı değiştiğinde (örneğin ilişkisel bir veritabanından NoSQL'e geçişte) yalnızca VT sınıfı değişir, iş mantığı sınıfları etkilenmez.
- İş mantığı katmanı ve veritabanı katmanından oluşan katmanlı bir yapı ortaya çıkar. Bağlaşım ile uyum çatıştığında daha katmanlı ve derli toplu çözüm seçilir.

Soru 3: Metot-alan erişim deseninden uyum (ekrandan, 01:50:00 ve 01:51:10)

Bir sınıfta üç alan (a1, a2, a3) ve dört metot (m1-m4) vardır. Erişimler: m1→a1, m1→a2, m2→a1, m2→a2, m3→a2, m4→a3. Sınıfın uyumunu hesaplayın. Sınıf {a1, a2, m1, m2, m3} ve {a3, m4} olarak ikiye bölünürse her sınıfın uyumu ve ortalama uyum ne olur?

Derste verilen çözüm:

- Tek sınıf: olası en çok erişim $3 \times 4 = 12$, gerçekleşen 6, uyum $6/12 = 0{,}5$ (%50).
- Sınıf I: olası en çok $2 \times 3 = 6$, gerçekleşen 5, uyum $5/6 \approx 0{,}83$. Sınıf II: $1/1 = 1$ (%100).
- Ortalama: $(5/6 + 1)/2 \approx 0{,}917$, yani yaklaşık %92 (hoca yuvarlanmış değerlerle %91,5 buldu). Sınıfı sorumluluklarına göre bölmek uyumu belirgin biçimde artırır.

Dersin Toparlaması

Bu ilk derste dersin iki kısmı tanıtıldı: Dörtlü Çete'nin tasarım kalıpları ve kod kusurları ile refactoring. Ardından tasarım kalıplarının neden çalışıldığı açıklandı. İyi tasarımın amacı kaliteli yazılımdır; kalıplar özellikle programcıya yönelik iç kalite özelliklerini (yeniden kullanılabilirlik, esneklik, anlaşılabilirlik, bakım kolaylığı) iyileştirir. Yazılım projelerinin başarı oranlarının düşük olması ve hata düzeltme bedelinin hayat döngüsünde üstel artması, iyi tasarımı denetlenebilir bir başarı etkeni yapar. İyi tasarımın iki temel ölçütü olan bağlaşım ve uyum sayısal örneklerle tartışıldı. Bağlaşım için toplam, maksimum ve dolaylı ölçüler hesaplandı. Uyum için metot-alan erişim deseni kullanıldı ve sınıfı bölmenin uyumu nasıl artırdığı gösterildi. Veritabanı örneği, iki ilkenin çatıştığı durumda katmanlı ve uyumlu çözümün seçilmesi gerektiğini gösterdi.

Son bölümde tasarım kalıbının tanımı, dört bileşeni (isim, sorun, çözüm, tartışma/sonuçlar), kalıplar arası ilişkiler, kazanımları ve GoF'un yaratımsal-yapısal-davranışsal sınıflandırması anlatıldı. İlk örnek kalıp olarak MVC işlendi: veri, gösterim ve denetimin ayrılması, asgari bağımlılıklar ve Observer ile Strategy'ye bağlanan çeşitlemeler. Bir sonraki derste GoF'un anlatım sırasına uygun olarak yaratımsal kalıplara geçilecek ve Abstract Factory ile başlanacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.